Week 1, Day 2: JVM Memory, Garbage Collection and Strings

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: JVM memory model and GC | DSA: Strings

Today's Schedule

- **0:00–0:45** Study JVM memory areas and class loading (sections 1 and 2).
- **0:45–1:30** Study garbage collection and the collectors (sections 3 and 4).
- **1:30–2:30** DSA: solve the 4 string problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **2:30–3:00** Study memory leaks and tuning (sections 5 and 6), then answer the interview questions aloud (section 8) and add 10 flashcards.
- **Optional (30 min):** write a small program that leaks memory with a static list, then watch it in VisualVM or jstat.

1. JVM Runtime Memory Areas

- **Heap:** all objects and arrays live here, shared by all threads. Split into the young generation (Eden plus two Survivor spaces, S0 and S1) and the old generation.
- **Thread stack:** one per thread. Holds frames with local variables, the operand stack and return addresses. Primitives and object references live here, the objects themselves live on the heap. Size is set with -Xss.
- **Metaspace:** class metadata, method data and constant pool. It lives in native memory and replaced PermGen in Java 8. It grows dynamically unless limited with -XX:MaxMetaspaceSize.
- **PC register and native method stack:** per-thread. The PC register tracks the current instruction, the native stack serves native (JNI) calls.
- **Code cache:** compiled machine code produced by the JIT compiler.
- **String pool:** interned strings, stored on the heap since Java 7. String literals share one instance, while new String("a") creates a separate object.
- **Common sizing flags:** -Xms (initial heap), -Xmx (maximum heap). Setting them equal avoids heap resizing pauses in production.

Errors and what they mean

- **OutOfMemoryError: Java heap space:** heap is full. Cause is a leak or an undersized heap.
- **OutOfMemoryError: Metaspace:** too many classes loaded, often from classloader leaks or heavy dynamic proxy and reflection use.
- **StackOverflowError:** thread stack exhausted, usually from infinite or very deep recursion.
- **OutOfMemoryError: GC overhead limit exceeded:** the JVM spends almost all its time in GC and recovers very little memory.

2. Class Loading

- **Phases:** loading, then linking (verification, preparation, resolution), then initialization (static initializers run).
- **Class loaders:** Bootstrap (core Java classes), Platform (formerly Extension), Application (your classpath).
- **Parent delegation:** a loader first asks its parent to load a class, and only loads it itself if the parent cannot. This keeps core classes like java.lang.String from being replaced.
- A class is identified by its name plus its class loader. The same class loaded by two loaders is two different types, which is a common source of ClassCastException in app servers.

3. Garbage Collection Basics

- **Reachability:** an object is garbage when no chain of references leads to it from a GC root (thread stacks, static fields, JNI references).
- **Generational hypothesis:** most objects die young, so the heap is split by age. New objects start in Eden.
- **Minor GC:** collects the young generation. Survivors are copied between Survivor spaces and promoted to the old generation after reaching an age threshold.
- **Major / mixed / full GC:** collect the old generation or the whole heap. Full GC is the most expensive and the one to avoid.
- **Stop-the-world (STW):** application threads pause during parts of GC. Your goal in tuning is to keep these pauses short and rare.
- **Algorithms:** mark-sweep (mark live, free dead), mark-compact (also move survivors together to remove fragmentation), copying (move live objects to a fresh area).

Reference types

- **Strong:** normal references, never collected while reachable.

- **Soft:** collected only under memory pressure, useful for caches.
- **Weak:** collected at the next GC if only weakly reachable (WeakHashMap).
- **Phantom:** used for post-mortem cleanup actions.

4. The Collectors

- **Serial:** single-threaded, for tiny heaps and small containers.
- **Parallel:** multi-threaded young and old collection, maximizes throughput, longer pauses. Default up to Java 8.
- **G1 (Garbage First):** default since Java 9. Heap is divided into equal regions, collects the regions with the most garbage first, and aims for a pause target set with -XX:MaxGCPauseMillis (default 200 ms). Good general-purpose choice for most Spring Boot services.
- **ZGC:** low-latency collector that does most work concurrently with the application, with very short pauses even on large heaps. Production-ready since Java 15, and generational mode arrived in Java 21. Enable with -XX:+UseZGC.
- **Shenandoah:** another low-pause concurrent collector, available in some JDK builds.
- **CMS:** removed in Java 14. Mention only as history.

Choosing

- Throughput batch jobs: Parallel.
- Typical web services: G1.
- Strict latency targets and large heaps: ZGC (or Shenandoah).

5. Memory Leaks in Java

Java has garbage collection, but leaks still happen when objects stay reachable by mistake.

- Static collections (a static Map or List) that only ever grow.
- Caches with no size limit or eviction. Use Caffeine or Guava with maximum size and expiry.
- Listeners and callbacks registered but never removed.
- ThreadLocal values in thread pools that are not removed. Pool threads live forever, so the value stays. Always call remove() in a finally block.
- Unclosed resources: streams, connections, result sets. Use try-with-resources.
- Inner classes and lambdas holding an implicit reference to the outer object.
- Classloader leaks after redeployments, which show up as Metaspace growth.

6. Diagnosing and Tuning (Connect This to Your Experience)

Your resume mentions production debugging and root-cause analysis, so prepare a story using this workflow.

- **Turn on safety nets:** -XX:+HeapDumpOnOutOfMemoryError and GC logging with -Xlog:gc* (Java 9+).
- **Watch live:** jstat -gcutil for GC activity, jcmd or jps to find processes, VisualVM or Java Flight Recorder (JFR) for profiling.
- **Thread problems:** jstack or jcmd Thread.print to capture thread dumps and find deadlocks and blocked threads.
- **Heap problems:** take a dump with jmap or jcmd GC.heap_dump, open it in Eclipse MAT, and look at the dominator tree and "leak suspects" report.
- **Typical tuning steps:** right-size the heap with -Xms and -Xmx, pick the collector for your latency goal, and in containers make sure the JVM respects the container memory limit.
- **Prepare a story:** symptom (slow responses or restarts), tool you used, root cause, fix, and measurable result.

7. DSA: Strings (4 Problems)

Problem 1: Valid Anagram

- **Idea:** count characters with an int[26] array. Increment for the first string, decrement for the second, and check that every count is zero.
- **Complexity:** O(n) time, O(1) space.
- **Follow-up to prepare:** what if the input has Unicode characters? Use a HashMap of counts instead of a fixed array.

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

Problem 2: Group Anagrams

- **Idea:** give every word a canonical key and group words with the same key in a HashMap. The key is either the sorted characters, or a 26-slot count signature.
- **Complexity:** O(n x k log k) with sorting, O(n x k) with count signatures, where k is the word length.
- **Edge cases:** empty string, single words, words with repeated letters.

Problem 3: Longest Substring Without Repeating Characters

- **Idea:** sliding window. Keep a map from character to its last index. When you see a repeat inside the window, move the left edge to one past that previous index.
- **Complexity:** O(n) time, O(min(n, alphabet)) space.
- **Edge cases:** empty string, all identical characters, repeat that lies before the window start (do not move left backwards).

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (last.containsKey(c) && last.get(c) >= left) {
            left = last.get(c) + 1;
        }
        last.put(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Problem 4: Valid Palindrome

- **Idea:** two pointers from both ends. Skip characters that are not letters or digits, compare the rest ignoring case.
- **Complexity:** O(n) time, O(1) space.
- **Edge cases:** empty string or only punctuation (true), mixed case, digits.
- **Hint:** use Character.isLetterOrDigit and Character.toLowerCase.

8. Interview Questions to Answer Aloud

1. Explain the JVM memory areas. What lives on the heap and what lives on the stack?
2. What replaced PermGen in Java 8, and where does it live?

3. What is the difference between a minor GC and a full GC?
4. How does G1 differ from the Parallel collector? Why is it the default now?
5. When would you choose ZGC?
6. What is a GC root? Give four examples.
7. How does class loading work, and what is parent delegation?
8. Can a Java application have a memory leak? Give three examples.
9. Why must you call remove() on a ThreadLocal in a thread pool?
10. What is the difference between OutOfMemoryError and StackOverflowError?
11. Your service is slowly getting slower and restarts every few days. How do you find the cause?
12. How does the String pool work, and what does intern() do?
13. Strong, soft, weak and phantom references: when is each useful?

9. End-of-Day Checklist

- [ ] I can draw the JVM memory areas and say what each one holds.
- [ ] I can explain the generational heap and minor vs full GC.
- [ ] I can say when to pick G1, Parallel and ZGC.
- [ ] I can name five causes of memory leaks and the tool I would use to find one.
- [ ] I solved all 4 string problems and can state their time and space complexity.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Day 3): Collections deep dive, Comparable vs Comparator, immutability, plus hashing problems.